

1. FCFS Scheduling algorithm

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

```
v_santosh@DESKTOP-E34M7RI:~$ nano FCFS.c
v_santosh@DESKTOP-E34M7RI:~$ gcc FCFS.c -o FCFS
v_santosh@DESKTOP-E34M7RI:~$ ./FCFS
Enter number of processes: 5
P1: 3
P2: 8
P3: 7
P4: 15
P5: 45

Process  BT      WT      AT
P1       3        0        3
P2       8        3       11
P3       7       11       18
P4      15       18       33
P5      45       33       78
```

2. SJF_NONpreemptive

```

#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Sort burst times
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (bt[i] > bt[j]) {
                temp = bt[i];
                bt[i] = bt[j];
                bt[j] = temp;
            }
        }
    }

    // Waiting time
    wt[0] = 0;

    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");

    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n",
            i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}

```

```

v_santosh@DESKTOP-E34M7RI:~$ nano SJF.c
v_santosh@DESKTOP-E34M7RI:~$ gcc SJF.c -o SJF
v_santosh@DESKTOP-E34M7RI:~$ ./SJF
Enter number of processes: 5
Enter burst time:
P1: 9
P2: 8
P3: 5
P4: 5
P5: 6

Process BT      WT      TAT
P1      5       0       5
P2      5       5      10
P3      6      10      16
P4      8      16      24
P5      9      24      33

```

3. Priority

```

#include <stdio.h>

int main() {
    int n, i, j;
    int bt[20], priority[20], wt[20], tat[20];
    int temp;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time and priority:\n");

    for (i = 0; i < n; i++) {
        printf("P%d Burst Time: ", i + 1);
        scanf("%d", &bt[i]);

        printf("P%d Priority: ", i + 1);
        scanf("%d", &priority[i]);
    }

    // Sort according to priority
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {

            if (priority[i] > priority[j]) {

                temp = priority[i];
                priority[i] = priority[j];
                priority[j] = temp;
            }
        }
    }
}

```

```

        temp = bt[i];
        bt[i] = bt[j];
        bt[j] = temp;
    }
}

// Waiting time
wt[0] = 0;

for (i = 1; i < n; i++) {
    wt[i] = wt[i - 1] + bt[i - 1];
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tPriority\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t\t%d\t%d\n",
        i + 1, bt[i], priority[i], wt[i], tat[i]);
}

return 0;
}

```

```

v_santosh@DESKTOP-E34M7RI:~$ nano Priority.c
v_santosh@DESKTOP-E34M7RI:~$ gcc Priority.c -o Priority
v_santosh@DESKTOP-E34M7RI:~$ ./Priority
Enter number of processes: 5
Enter burst time and priority:
P1 Burst Time: 8
P1 Priority: 5
P2 Burst Time: 6
P2 Priority: 8
P3 Burst Time: 2
P3 Priority: 6
P4 Burst Time: 8
P4 Priority:
8
P5 Burst Time: 6
P5 Priority: 5

```

Process	BT	Priority	WT	TAT
P1	8	5	0	8
P2	6	5	8	14
P3	2	6	14	16
P4	8	8	16	24
P5	6	8	24	30

4. ROUND ROBIN

```

#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");

    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);

        rem_bt[i] = bt[i];
        wt[i] = 0;
    }
}

```

```

printf("Enter time quantum: ");
scanf("%d", &quantum);

// Round Robin
while (1) {

    done = 1;

    for (i = 0; i < n; i++) {

        if (rem_bt[i] > 0) {

            done = 0;

            if (rem_bt[i] > quantum) {
                time = time + quantum;
                rem_bt[i] = rem_bt[i] - quantum;
            }
            else {
                time = time + rem_bt[i];

                wt[i] = time - bt[i];

                rem_bt[i] = 0;
            }
        }
    }

    if (done == 1)
        break;
}

// Turnaround time
for (i = 0; i < n; i++) {
    tat[i] = wt[i] + bt[i];
}

printf("\nProcess\tBT\tWT\tTAT\n");

for (i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\n",
        i + 1, bt[i], wt[i], tat[i]);
}

return 0;
}

```

```
v_santosh@DESKTOP-E34M7RI:~$ nano Round.c
v_santosh@DESKTOP-E34M7RI:~$ gcc Round.c -o Round
v_santosh@DESKTOP-E34M7RI:~$ ./Round
```

Enter number of processes: 5

Enter burst time:

P1: 2

P2: 2

P3: 2

P4: 2

P5: 2

Enter time quantum: 2

Process	BT	WT	TAT
P1	2	0	2
P2	2	2	4
P3	2	4	6
P4	2	6	8
P5	2	8	10